

Python Programming Basic

Tutorial References

<https://www.tutorialspoint.com/python/index.htm>

<https://www.javatpoint.com/python-tutorial>

APPLICATION

An application, often referred to as an "app," is a type of software designed to perform a specific function directly for the user or, in some cases, for another application program. Applications can be as simple as a calculator app or as complex as a full-fledged enterprise resource planning system.

There are several types of applications, categorized based on their functionality, the platforms they run on, and their intended use. Here's an overview of the main types:

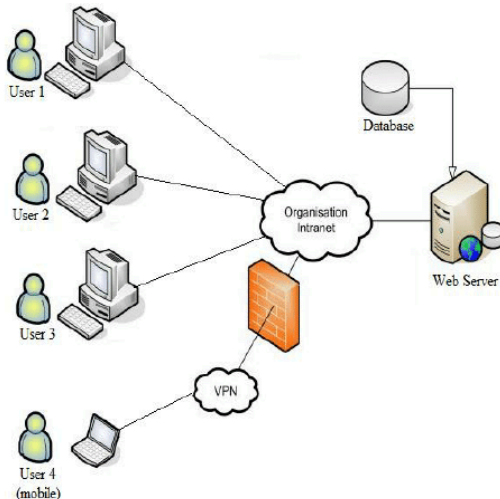


1. Desktop Applications (Stand Alone Application)

- **Definition:** Software programs that run on a desktop or laptop computer.
- **Examples:** Microsoft Office (Word, Excel), Adobe Photoshop, VLC Media Player.

2. Web Applications (Client - Server)

- **Definition:** Applications that run in a web browser and can be accessed through the internet.



- **Examples:** Google Docs, Gmail, Facebook, Trello.

3. Mobile Applications

- **Definition:** Applications designed to run on mobile devices such as smartphones and tablets.
- **Examples:** Instagram, WhatsApp, Uber, Spotify.

4. Enterprise Applications

- **Definition:** Large-scale software systems designed to operate in a corporate environment to satisfy the needs of an organization rather than individual users.
- **Examples:** Customer Relationship Management (CRM) systems, Enterprise Resource Planning (ERP) systems, Human Resource Management Systems (HRMS).

5. Utility Applications

- **Definition:** Software designed to help manage and tune the computer hardware, operating system, or application software by performing a single task or a small range of tasks.
- **Examples:** Antivirus programs, disk cleanup tools, backup software.

6. Database Applications

- **Definition:** Software that interacts with databases to capture and analyze data.
- **Examples:** MySQL Workbench, Oracle Database, Microsoft SQL Server.

7. Embedded Applications

- **Definition:** Software designed to operate hardware with minimal user interaction, often running on devices other than traditional computers.
- **Examples:** Firmware for appliances, software for smart TVs, automotive systems.

8. Educational Applications

- **Definition:** Software designed to facilitate learning and provide educational content.
- **Examples:** Duolingo, Khan Academy, Google Classroom.

9. Gaming Applications

- **Definition:** Software created for entertainment and recreation purposes.
- **Examples:** Fortnite, Candy Crush Saga, The Sims.

10. Multimedia Applications

- **Definition:** Software designed for creating or consuming multimedia content, such as audio, video, and graphics.
- **Examples:** Adobe Premiere Pro, iTunes, VLC Media Player.

11. Productivity Applications

- **Definition:** Software aimed at increasing productivity by assisting with tasks like word processing, spreadsheet analysis, and presentations.
- **Examples:** Microsoft Office Suite, Google Workspace, Slack.

12. Communication Applications

- **Definition:** Software that facilitates communication between users.
- **Examples:** Zoom, Skype, Microsoft Teams, Slack.

13. Financial Applications

- **Definition:** Software designed to manage personal or business finances.
- **Examples:** QuickBooks, Mint, PayPal, Robinhood.

Each type of application serves a unique purpose and is designed with specific user needs and technological capabilities in mind. Applications continue to evolve, leveraging advancements in technology to provide enhanced functionality and improved user experiences.

Introduction to 3-Tier Architecture in Application Development

Three-tier architecture is a well-established software architecture pattern used in application development. It is designed to separate the concerns of an application into three distinct layers, improving scalability, maintainability, and reusability. The three tiers are:

1. **Presentation Tier (Front End) ----> User Interface ---> CUI / GUI / MUI
(Designing)**
2. **Logic Tier (Back END) ---- Prog. Lang**
3. **Data Tier -----> Database Mgt -----> DBA**

1. Presentation Tier

Description: The presentation tier is the topmost layer of the application that interacts directly with the users. It handles the user interface and displays information to the user. This layer is responsible for capturing user inputs and presenting the outputs back to the user.

Components:

- Web Browsers
- Mobile Apps
- Desktop Applications

Technologies:

- HTML, CSS, JavaScript for web interfaces
- Swift, Kotlin for mobile interfaces
- WPF, WinForms for desktop interfaces

Pros:

- Enhances user experience by providing a user-friendly interface.

- Separates the UI code from business logic and data access code.

Cons:

- Requires careful design to ensure compatibility across different devices and platforms.

Example: In an e-commerce application, the presentation tier would include the website where users browse products, add items to the cart, and make purchases.

WHAT IS FRONT END (Presentation Tier)

The front end refers to the part of a software application or website that users interact with directly. It is the interface and user experience (UX) aspect of an application, encompassing everything that users see, click, and interact with. The front end is also known as the client-side of a web application.

Key Components of Front End

1. **HTML (HyperText Markup Language):**
 - Defines the structure and content of web pages.
 - Uses elements like headings, paragraphs, lists, links, images, and more to organize content.
2. **CSS (Cascading Style Sheets):**
 - Controls the presentation, layout, and styling of web pages.
 - Allows developers to apply styles such as fonts, colors, spacing, and responsiveness.
3. **JavaScript:**
 - Adds interactivity and dynamic behavior to web pages.
 - Enables functionalities such as form validation, content updates without refreshing the page, animations, and handling user events (clicks, mouseovers, etc.).
4. **Frameworks and Libraries:**
 - **CSS Frameworks:** Bootstrap, Tailwind CSS, Bulma (help streamline responsive design and styling).
 - **JavaScript Libraries:** jQuery (simplifies DOM manipulation and event handling).
 - **JavaScript Frameworks:** React, Angular, Vue.js (provide structured approaches to building complex user interfaces).
5. **Front End Tools:**
 - **Version Control Systems:** Git (for tracking changes in the code).
 - **Build Tools:** Webpack, Gulp, Grunt (for automating tasks like minification, compilation, and bundling).
 - **Package Managers:** npm, Yarn (for managing libraries and dependencies).

Front End Development Process

1. **Design:** Creating wireframes, mockups, and prototypes to outline the visual and functional aspects of the application.
2. **Development:** Writing HTML, CSS, and JavaScript code to implement the design and functionality.
3. **Testing:** Ensuring the interface works correctly across different browsers and devices (cross-browser and cross-device compatibility).
4. **Optimization:** Improving performance by optimizing assets (images, scripts) and ensuring fast load times.
5. **Deployment:** Making the front end live by hosting it on a server and connecting it to the back end (if applicable).

Importance of Front End Development

- **User Experience:** A well-designed front end enhances user satisfaction by providing a seamless and intuitive interface.
- **Accessibility:** Ensuring that the application is usable by people with various disabilities.
- **Performance:** Optimizing load times and responsiveness, especially important for retaining users.
- **Branding:** Consistent and appealing design reinforces brand identity and trust.

In summary, the front end is a critical aspect of web development, focusing on how users interact with

2. Logic Tier

Description: The logic tier, also known as the business logic or application tier, processes the business logic and data transformation. This layer acts as an intermediary between the presentation and data tiers, ensuring that business rules are correctly applied.

Components:

- Application Servers
- Web Services
- Business Logic Components

Technologies:

- Java, C#, Python for server-side logic
- Frameworks like Spring, .NET, Django

Pros:

- Centralizes business logic, making it easier to maintain and update.
- Enhances security by preventing direct access to the data tier.

Cons:

- Can become complex and require careful design to manage dependencies and interactions.

Example: In an e-commerce application, the logic tier would handle operations such as user authentication, product search, order processing, and payment validation.

What is Back end

the term "back end" refers to the server-side part of a web application or software system. It encompasses the components and functionality that operate behind the scenes and are not visible to the user. The back end is responsible for processing business logic, database interactions, authentication, authorization, server configuration, and any other operations required to fulfill requests from the front end (client-side) and return the appropriate data or response.

Key components of the back end include:

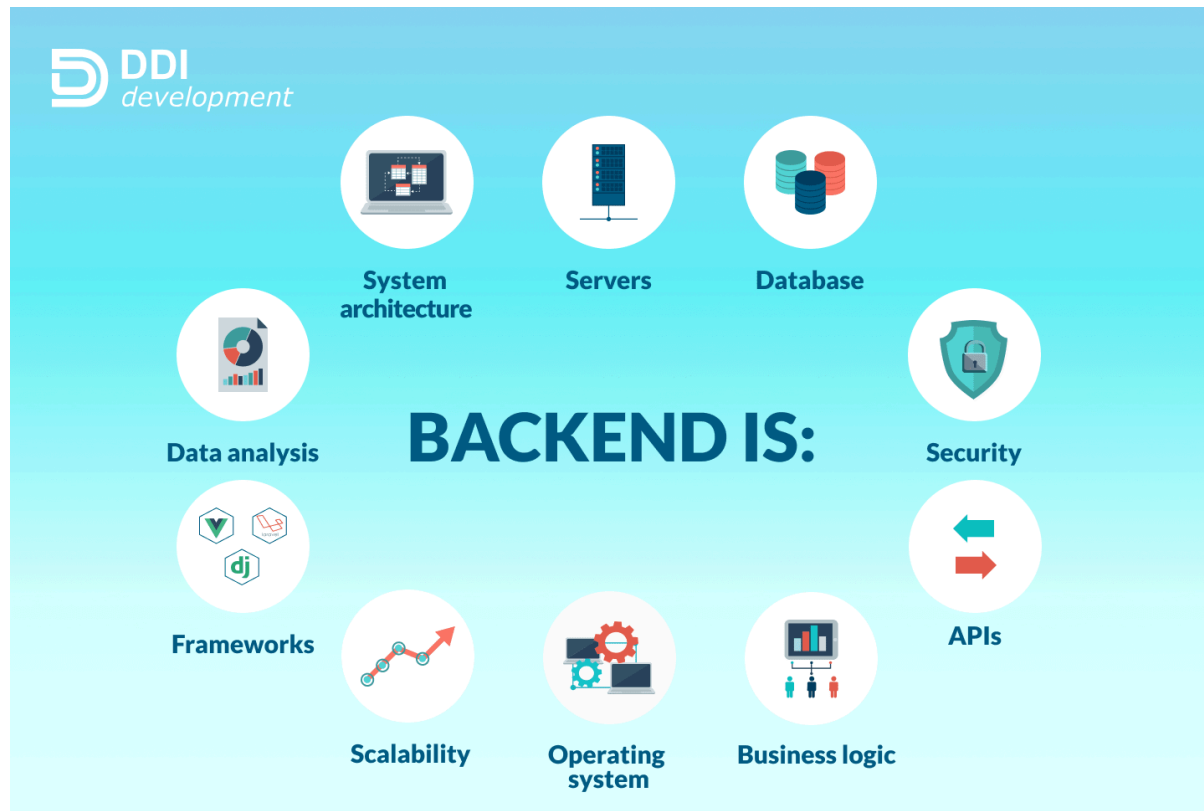
1. **Server:** The computer or set of computers that host the back end application and handle requests from clients (users' devices).
2. **Database:** The system used to store, manage, and retrieve data. Common types of databases include SQL databases (like MySQL, PostgreSQL) and NoSQL databases (like MongoDB, Cassandra).
3. **Application Logic:** The code that processes input from the front end, performs necessary computations, and interacts with the database. This is usually written in server-side programming languages such as Python, Java, Ruby, PHP, Node.js, or C#.
4. **API (Application Programming Interface):** A set of rules and protocols for building and interacting with software applications. The back end often exposes APIs that the front end can use to send and retrieve data.
5. **Authentication and Authorization:** Processes that ensure users are who they say they are (authentication) and determine what they are allowed to do (authorization).
6. **Server Configuration and Management:** Involves setting up and maintaining the server environment, including operating systems, web servers (like Apache, Nginx), and deployment processes.

Example Scenario

Consider an e-commerce website:

- **Front end:** The part that users interact with, such as viewing products, adding them to a shopping cart, and making a purchase.
- **Back end:** Manages the product database, processes user orders, handles payment transactions, manages user sessions, and communicates with the front end to provide necessary data (e.g., product details, user order history).

The back end is crucial for the overall functionality of any web application, ensuring that data flows smoothly between the user interface and the database, and that business rules are correctly implemented.



3. Data Tier

Description: The data tier, also known as the database or storage tier, is responsible for storing and retrieving data. This layer interacts with databases and file storage systems to manage data persistence.

Components:

- Database Management Systems (DBMS)
- Data Access Components
- Storage Systems

Technologies:

- SQL Databases like MySQL, PostgreSQL
- NoSQL Databases like MongoDB, Cassandra
- ORM tools like Hibernate, Entity Framework

Pros:

- Provides a centralized location for data management.
- Ensures data consistency and integrity.

Cons:

- Requires robust security measures to protect sensitive data.
- Can become a bottleneck if not properly optimized.

Example: In an e-commerce application, the data tier would store information about products, user accounts, orders, and transaction history.

What is a Database?

A database is an organized collection of data, typically stored and accessed electronically from a computer system. Databases are designed to efficiently store, retrieve, and manage data. They are an essential component of modern software applications, enabling structured storage and easy access to vast amounts of information.

Key Components of a Database

1. **Data:**
 - The actual information or facts that are stored. This can include text, numbers, images, videos, and more.
2. **Schema:**
 - The structure that defines how the data is organized, including tables, fields, data types, and relationships between tables.
3. **Database Management System (DBMS):**
 - The software that interacts with the database, allowing users to create, read, update, and delete data. Popular DBMSs include MySQL, PostgreSQL, Microsoft SQL Server, and Oracle.
4. **Queries:**
 - Requests made to the database to retrieve or manipulate data. Structured Query Language (SQL) is commonly used to perform queries.
5. **Indexes:**
 - Structures that improve the speed of data retrieval operations by providing quick access to rows in a table based on the values of one or more columns.
6. **Transactions:**
 - A sequence of operations performed as a single logical unit of work. Transactions ensure data integrity and consistency.

Types of Databases

1. Relational Databases:

- Use tables (relations) to store data. Tables consist of rows and columns, and relationships between tables are established using foreign keys. Examples: MySQL, PostgreSQL, SQLite, Microsoft SQL Server.

2. NoSQL Databases:

- Designed for specific data models and include key-value stores, document stores, wide-column stores, and graph databases. They are often used for big data and real-time web applications. Examples: MongoDB (document store), Cassandra (wide-column store), Redis (key-value store), Neo4j (graph database).

3. NewSQL Databases:

- Combine the scalability of NoSQL with the ACID (Atomicity, Consistency, Isolation, Durability) properties of traditional relational databases. Examples: Google Spanner, CockroachDB.

4. In-Memory Databases:

- Store data in the main memory (RAM) rather than on disk, providing faster data access and processing. Examples: Redis, Memcached.

5. Cloud Databases:

- Delivered as a service and accessed over the internet, providing scalability and flexibility. Examples: Amazon RDS, Google Cloud SQL, Microsoft Azure SQL Database.

Common Database Operations

1. Create:

- Adding new data to the database. For example, inserting a new row into a table.

2. Read:

- Retrieving data from the database. This includes running queries to fetch specific information.

3. Update:

- Modifying existing data in the database. For example, changing a value in a table.

4. Delete:

- Removing data from the database.

Advantages of Using Databases

1. Data Integrity and Consistency:

- Ensures that the data remains accurate and consistent through the use of constraints and transactions.

2. Efficient Data Management:

- Allows for efficient storage, retrieval, and management of data, especially when dealing with large volumes of information.

3. **Data Security:**
 - Provides mechanisms to control access to data, ensuring that only authorized users can perform certain operations.
4. **Data Sharing:**
 - Enables multiple users and applications to access and use the same data concurrently.
5. **Backup and Recovery:**
 - Facilitates data backup and recovery processes, ensuring data is not lost in case of failures.

Real-World Applications of Databases

1. **E-Commerce:**
 - Managing product inventories, customer information, orders, and payments.
2. **Banking:**
 - Handling transactions, customer accounts, loan processing, and compliance reporting.
3. **Healthcare:**
 - Storing patient records, appointment scheduling, and medical research data.
4. **Education:**
 - Managing student information, course registrations, grades, and academic research.
5. **Social Media:**
 - Handling user profiles, posts, messages, and interactions.

Conclusion

Databases are fundamental to the functioning of modern software applications and business processes. They provide a structured way to store, manage, and retrieve large amounts of data efficiently. Understanding the basics of databases, their types, and their operations is crucial for anyone involved in software development, data analysis, or IT management.

- The structure that defines how the data is organized, including tables, fields, data types, and relationships between tables.
2. **Database Management System (DBMS):**
 - The software that interacts with the database, allowing users to create, read, update, and delete data. Popular DBMSs include MySQL, PostgreSQL, Microsoft SQL Server, and Oracle.
 3. **Queries:**
 - Requests made to the database to retrieve or manipulate data. Structured Query Language (SQL) is commonly used to perform queries.
 4. **Indexes:**

- Structures that improve the speed of data retrieval operations by providing quick access to rows in a table based on the values of one or more columns.
5. **Transactions:**
- A sequence of operations performed as a single logical unit of work. Transactions ensure data integrity and consistency.

Types of Databases

1. **Relational Databases:**
 - Use tables (relations) to store data. Tables consist of rows and columns, and relationships between tables are established using foreign keys. Examples: MySQL, PostgreSQL, SQLite, Microsoft SQL Server.
2. **NoSQL Databases:**
 - Designed for specific data models and include key-value stores, document stores, wide-column stores, and graph databases. They are often used for big data and real-time web applications. Examples: MongoDB (document store), Cassandra (wide-column store), Redis (key-value store), Neo4j (graph database).
3. **NewSQL Databases:**
 - Combine the scalability of NoSQL with the ACID (Atomicity, Consistency, Isolation, Durability) properties of traditional relational databases. Examples: Google Spanner, CockroachDB.
4. **In-Memory Databases:**
 - Store data in the main memory (RAM) rather than on disk, providing faster data access and processing. Examples: Redis, Memcached.
5. **Cloud Databases:**
 - Delivered as a service and accessed over the internet, providing scalability and flexibility. Examples: Amazon RDS, Google Cloud SQL, Microsoft Azure SQL Database.

Common Database Operations

1. **Create:**
 - Adding new data to the database. For example, inserting a new row into a table.
2. **Read:**
 - Retrieving data from the database. This includes running queries to fetch specific information.
3. **Update:**
 - Modifying existing data in the database. For example, changing a value in a table.
4. **Delete:**
 - Removing data from the database.

Advantages of Using Databases

1. **Data Integrity and Consistency:**
 - Ensures that the data remains accurate and consistent through the use of constraints and transactions.
2. **Efficient Data Management:**
 - Allows for efficient storage, retrieval, and management of data, especially when dealing with large volumes of information.
3. **Data Security:**
 - Provides mechanisms to control access to data, ensuring that only authorized users can perform certain operations.
4. **Data Sharing:**
 - Enables multiple users and applications to access and use the same data concurrently.
5. **Backup and Recovery:**
 - Facilitates data backup and recovery processes, ensuring data is not lost in case of failures.

Real-World Applications of Databases

1. **E-Commerce:**
 - Managing product inventories, customer information, orders, and payments.
2. **Banking:**
 - Handling transactions, customer accounts, loan processing, and compliance reporting.
3. **Healthcare:**
 - Storing patient records, appointment scheduling, and medical research data.
4. **Education:**
 - Managing student information, course registrations, grades, and academic research.
5. **Social Media:**
 - Handling user profiles, posts, messages, and interactions.

Conclusion

Databases are fundamental to the functioning of modern software applications and business processes. They provide a structured way to store, manage, and retrieve large amounts of data efficiently. Understanding the basics of databases, their types, and their operations is crucial for anyone involved in software development, data analysis, or IT management.

Benefits of 3-Tier Architecture

1. **Scalability:** Each tier can be scaled independently based on the load. For instance, the presentation tier can be scaled horizontally by adding more web servers.
2. **Maintainability:** By separating the concerns into distinct layers, the system becomes easier to maintain. Changes in one layer typically do not impact the other layers.

3. **Reusability:** The business logic can be reused across different applications. For example, the same logic tier can serve a web application and a mobile app.
4. **Improved Security:** Since the data tier is not directly exposed to the client, it provides an additional layer of security. The logic tier can validate and sanitize inputs before they reach the data tier.
5. **Flexibility:** Different teams can work on different tiers simultaneously, using technologies that are best suited for each layer. This can speed up the development process.

Conclusion

Three-tier architecture is a robust and versatile pattern used in application development to create scalable, maintainable, and secure applications. By separating the application into presentation, logic, and data tiers, developers can manage complexity more effectively and ensure that each layer performs its role efficiently. This architecture is widely used in various applications, from simple web applications to complex enterprise systems

Day 1: Introduction to Full Stack Development

- **Session Topic:** Overview of Full Stack Development
- **Description:**
 - **Understanding Full Stack Development:**
 - Definition and significance of full stack development.
 - Roles and responsibilities of a full stack developer.
 - **Components of Web Applications:**
 - Frontend: User interface and experience.
 - Backend: Server-side logic and integration.
 - Database: Data storage and management.
 - **Introduction to Python in Web Development:**
 - Why Python is a preferred language for web development.
 - Overview of popular Python web frameworks: Django and Flask.

Day 2: Python Fundamentals for Web Development

- **Session Topic:** Core Python Concepts
- **Description:**
 - **Python Basics:**
 - Setting up the Python environment.
 - Writing and executing Python scripts.
 - **Variables and Data Types:**
 - Numbers, strings, lists, tuples, dictionaries, and sets.
 - **Control Structures:**
 - Conditional statements (**if**, **elif**, **else**).
 - Looping constructs (**for**, **while** loops).

- **Functions:**
 - Defining and calling functions.
 - Understanding scope and lifetime of variables.
- **Object-Oriented Programming (OOP):**
 - Classes and objects.
 - Inheritance, polymorphism, encapsulation, and abstraction.

Day 3: Web Development Basics (HTML, CSS, Bootstrap)

- **Session Topic:** Building the Frontend
- **Description:**
 - **HTML5:**
 - Structure of an HTML document.
 - Semantic elements (`<header>`, `<footer>`, `<article>`, `<section>`).
 - Forms and input elements.
 - **CSS3:**
 - Styling text and layouts.
 - Box model, positioning, and flexbox.
 - **Bootstrap Framework:**
 - Introduction to responsive design.
 - Utilizing Bootstrap components and grid system for rapid UI development.

Day 4: JavaScript Essentials & DOM Manipulation

- **Session Topic:** Enhancing Interactivity
- **Description:**
 - **JavaScript Basics:**
 - Variables, data types, and operators.
 - Functions and scope.
 - **Document Object Model (DOM):**
 - Selecting and manipulating DOM elements.
 - Event handling (click, hover, etc.).
 - **Fetch API:**
 - Making asynchronous requests to servers.
 - Handling JSON data.

Day 5: Introduction to Django/Flask & MVC Architecture

- **Session Topic:** Backend Frameworks and Architecture
- **Description:**
 - **Django vs. Flask:**
 - Overview and comparison of both frameworks.
 - Choosing the right framework based on project requirements.
 - **Setting Up a Project:**
 - Installing Django/Flask.

- Creating and structuring a new project.
- **MVC (Model-View-Controller) Architecture:**
 - Understanding the flow of data and control.
 - Implementing models, views, and controllers in Django/Flask.

Day 6: Database Management (SQL & NoSQL)

- **Session Topic:** Data Storage Solutions
- **Description:**
 - **Relational Databases (SQL):**
 - Introduction to MySQL/PostgreSQL.
 - CRUD operations (Create, Read, Update, Delete).
 - Writing complex queries and joins.
 - **NoSQL Databases:**
 - Introduction to MongoDB.
 - Understanding document-based storage.
 - When to choose NoSQL over SQL.

Day 7: ORM & Database Integration

- **Session Topic:** Bridging the Application and Database
- **Description:**
 - **Object-Relational Mapping (ORM):**
 - Purpose and advantages of using ORM.
 - Setting up Django ORM / SQLAlchemy with Flask.
 - **Performing Database Operations:**
 - Defining models and schemas.
 - Querying the database using ORM methods.
 - Handling migrations and schema changes.

Day 8: User Authentication & Security

- **Session Topic:** Securing Web Applications
- **Description:**
 - **Authentication Mechanisms:**
 - Implementing user registration and login functionalities.
 - Password hashing and security best practices.
 - **Authorization:**
 - Role-based access control.
 - Protecting routes and resources.
 - **Security Measures:**
 - Implementing JWT (JSON Web Tokens) for stateless authentication.
 - Overview of OAuth for third-party authentications.
 - Introduction to Firebase Authentication as an alternative.

Day 9: REST API Development

- **Session Topic:** Building APIs for Communication
- **Description:**
 - **Understanding RESTful Services:**
 - Principles of REST architecture.
 - Designing RESTful endpoints.
 - **Implementing REST APIs:**
 - Using Django REST Framework (DRF) or Flask-RESTful.
 - Handling CRUD operations via API.
 - Serializing data for JSON responses.

Day 10: Frontend-Backend Integration

- **Session Topic:** Connecting the Dots
- **Description:**
 - **Setting Up the Frontend Framework:**
 - Introduction to React.js or using vanilla JavaScript.
 - Creating components and managing state.
 - **Making API Calls:**
 - Using `fetch` or Axios to send requests to the backend.
 - Handling responses and updating the UI accordingly.
 - **Cross-Origin Resource Sharing (CORS):**
 - Understanding and configuring CORS for seamless integration.

Day 11: Deployment & Hosting

- **Session Topic:** Bringing Applications to the Web
- **Description:**
 - **Preparing for Deployment:**
 - Setting up environment variables.
 - Configuring settings for production.
 - **Deployment Platforms:**
 - Deploying applications on AWS, Heroku, or Railway.
 - Setting up web servers (e.g., Nginx, Unicorn).
 - **Version Control:**
 - Using Git for source code management.
 - Collaborating with others using platforms like GitHub.

Day 12: WebSockets & Real-Time Features

- **Session Topic:** Adding Real-Time Capabilities
- **Description:**
 - **Understanding WebSockets:**
 - Difference between HTTP and WebSockets.

- Use cases for real-time communication.
- **Implementing WebSockets:**
 - Setting up Django Channels or Flask-SocketIO.
 - Building features like chat applications or live notifications.

Day 13: Performance Optimization & Caching

- **Session Topic:** Enhancing Application Efficiency
- **Description:**
 - **Performance Bottlenecks:**
 - Identifying common performance issues.
 - Tools and techniques for profiling applications.
 - **Caching Strategies:**
 - Implementing caching with Redis.
 - Caching templates, queries, and API responses.
 - **Database Optimization:**
 - Indexing tables for faster queries.
 - Optimizing ORM queries to reduce overhead.

Types of Programming Languages

Programming languages can be categorized based on their features, paradigms, and uses. Here are some common types of programming languages along with their descriptions:

1. Procedural Programming Languages

Description: Procedural programming languages are based on the concept of procedure calls, where statements are structured into procedures or functions. These languages follow a step-by-step approach to execute a set of instructions.

Characteristics:

- Follows a linear top-down approach
- Emphasizes the use of loops, conditionals, and subroutines
- Uses procedures/functions to break down tasks

Examples:

- **C:** A general-purpose language known for its performance and low-level memory manipulation.
 - **Pascal:** Used mainly for teaching programming and understanding structured programming concepts.
-

2. Object-Oriented Programming Languages

Description: Object-oriented programming (OOP) languages are based on the concept of "objects," which can contain data and code to manipulate the data. OOP emphasizes encapsulation, inheritance, and polymorphism.

Characteristics:

- Organizes code into objects and classes
- Promotes code reuse and modularity
- Uses concepts like inheritance, polymorphism, and encapsulation

Examples:

- **Java:** Widely used for building enterprise-scale applications and Android apps.
- **C++:** An extension of C that includes object-oriented features.
- **Python:** Supports multiple paradigms, including OOP, and is known for its simplicity and readability.

3. Functional Programming Languages

Description: Functional programming languages are based on mathematical functions. They emphasize the use of pure functions, immutability, and first-class functions.

Characteristics:

- Treats computation as the evaluation of mathematical functions
- Avoids changing-state and mutable data
- Supports higher-order functions and recursion

Examples:

- **Haskell:** Known for its strong static type system and lazy evaluation.
 - **Lisp:** One of the oldest functional languages, known for its flexibility and powerful macro system.
 - **Scala:** Combines functional and object-oriented programming, often used in big data applications.
-

4. Scripting Languages

Description: Scripting languages are used to write scripts that automate tasks. They are often interpreted rather than compiled and are used for writing small programs to perform repetitive tasks.

Characteristics:

- Typically interpreted, not compiled
- Easy to write and use for automating processes
- Often used in web development, system administration, and data analysis

Examples:

- **JavaScript:** The language of the web, used for client-side scripting.
 - **Python:** Versatile and widely used for scripting, automation, and data analysis.
 - **Ruby:** Known for its elegant syntax, often used in web development with the Ruby on Rails framework.
-

5. Markup Languages

Description: Markup languages are designed for the presentation of text and data. They use tags to define elements within a document.

Characteristics:

- Not programming languages in the traditional sense, but used to structure and format documents
- Uses tags to annotate text

Examples:

- **HTML:** The standard markup language for creating web pages.
 - **XML:** Used for storing and transporting data, with a focus on simplicity and generality.
 - **Markdown:** A lightweight markup language with plain text formatting syntax, often used for documentation.
-

6. Logic Programming Languages

Description: Logic programming languages are based on formal logic. Programs are written as sets of logical statements, and the execution is the process of proving queries against these statements.

Characteristics:

- Uses facts and rules to represent information and inference
- The main operation is logical inference

Examples:

- **Prolog:** Used in artificial intelligence and computational linguistics for its pattern matching and rule-based execution.
-

7. Concurrent Programming Languages

Description: Concurrent programming languages are designed to handle multiple computations simultaneously, enabling parallelism and efficient utilization of multi-core processors.

Characteristics:

- Supports concurrent execution of processes
- Uses constructs like threads, locks, and message passing

Examples:

- **Go:** Designed by Google, known for its simplicity and support for concurrent programming.
 - **Erlang:** Used in telecom and distributed systems for its concurrency and fault-tolerance capabilities.
-

8. Database Query Languages

Description: Database query languages are used to communicate with databases. They allow for the retrieval, manipulation, and management of data stored in a database.

Characteristics:

- Focuses on querying and manipulating data
- Provides operations for data definition, manipulation, and control

Examples:

- **SQL:** The standard language for relational database management systems.
 - **NoSQL Query Languages:** Used for non-relational databases, such as MongoDB's query language.
-

Conclusion

Programming languages come in various types, each designed to solve specific kinds of problems and suited for particular tasks. Understanding the different types helps developers choose the right language for their project, ensuring that they can leverage the strengths of the language to achieve their goals efficiently.

Types of Programming Languages Based on Execution and Platform

Programming languages can be categorized based on how they are executed and the platforms they support. Here are the main types:

Based on Execution:

1. Compiled Languages

Description: Compiled languages are translated directly into machine code by a compiler before execution. This machine code is specific to the target hardware and can be executed directly by the computer's CPU.

Characteristics:

- Typically offer faster execution times compared to interpreted languages.
- Requires a compilation step before running the program.
- Errors are detected at compile-time.

Examples:

- **C:** Widely used for system and application software.
- **C++:** An extension of C with object-oriented features.
- **Rust:** Known for its safety and performance.

Pros:

- High performance due to direct execution of machine code.
- Better optimization by the compiler.

Cons:

- Longer development cycle due to the compile step.
- Less portable; compiled code is specific to a platform.

2. Interpreted Languages

Description: Interpreted languages are executed line-by-line by an interpreter at runtime, without the need for a prior compilation step.

Characteristics:

- More flexible and easier to debug since they execute code directly.
- Slower execution compared to compiled languages due to the overhead of interpretation.
- Errors are detected at runtime.

Examples:

- **Python:** Known for its simplicity and versatility.
- **JavaScript:** Used primarily for client-side scripting on the web.
- **Ruby:** Valued for its elegant syntax and dynamic capabilities.

Pros:

- Easier to test and debug.
- Highly portable across different platforms without recompilation.

Cons:

- Slower execution due to runtime interpretation.
 - Performance overhead of the interpreter.
-

3. Just-In-Time (JIT) Compiled Languages

Description: JIT compiled languages combine features of both compiled and interpreted languages. They are initially compiled to an intermediate form, which is then interpreted or further compiled to machine code at runtime.

Characteristics:

- Balances between the development speed of interpreted languages and the performance of compiled languages.
- Intermediate code is portable across different platforms.

Examples:

- **Java:** Compiled to bytecode, which is executed by the Java Virtual Machine (JVM).

- **C#:** Compiled to Intermediate Language (IL), which is executed by the Common Language Runtime (CLR).

Pros:

- Improved runtime performance compared to purely interpreted languages.
- Platform independence due to the intermediate bytecode.

Cons:

- JIT compilation can introduce some overhead at runtime.
 - Requires a runtime environment (e.g., JVM, CLR).
-

Based on Platform :

1. Platform-Dependent Languages c/c++

Description: Platform-dependent languages produce code that runs on a specific type of hardware or operating system.

Characteristics:

- Compiled code is specific to the target platform.
- Often used for system programming or applications that need to leverage specific hardware features.

Examples:

- **Assembly Language:** Directly corresponds to machine code instructions for specific hardware.
- **Objective-C:** Primarily used for macOS and iOS development.

Pros:

- Can optimize performance for specific hardware.
- Direct access to system resources and hardware features.

Cons:

- Lack of portability; code needs significant changes to run on different platforms.
 - Higher maintenance effort for cross-platform support.
-

2. Platform-Independent Languages

Description: Platform-independent languages are designed to run on multiple types of hardware and operating systems without modification.

Source code

Byte Code --- >

M/c Code

Characteristics:

- Typically rely on a virtual machine or runtime environment to execute code.
- Promote write-once, run-anywhere capabilities.

Examples:

- **Java:** Runs on the Java Virtual Machine (JVM), making it platform-independent.
- **Python:** Can run on any system with a compatible interpreter.
- **JavaScript:** Runs in web browsers across different platforms.

Pros:

- High portability across different systems.
- Easier to maintain a single codebase for multiple platforms.

Cons:

- May not fully leverage platform-specific optimizations.
 - Performance might be less optimal compared to platform-specific languages.
-

Conclusion

Programming languages can be classified based on their execution methods and platform dependencies. Understanding these categories helps developers choose the appropriate language for their projects, considering factors such as performance, portability, and ease of development. Each type of language has its strengths and weaknesses, making them suitable for different scenarios and requirements in software development.

Compiler vs. Interpreter

Compilers and interpreters are two types of language processors used in programming languages to translate high-level code into machine code that a computer can understand and execute. They serve different purposes and have distinct characteristics, advantages, and disadvantages.

Compiler

Description: A compiler is a program that translates the entire source code of a programming language into machine code (binary code) before the program is run. The compiled code is a separate executable file that can be executed independently of the source code.

How It Works:

1. **Source Code:** The programmer writes the code in a high-level language.
2. **Compilation:** The compiler translates the entire source code into machine code.
3. **Executable:** The compiled machine code is saved as an executable file.
4. **Execution:** The executable file is run directly by the computer's CPU.

Pros:

- **Performance:** Compiled programs typically run faster than interpreted ones because the translation to machine code happens before execution.
- **Optimization:** Compilers can optimize the code during the compilation process, improving performance and efficiency.
- **Error Checking:** Many compilers perform extensive error checking and reporting, helping developers identify issues before execution.

Cons:

- **Compilation Time:** Compiling the entire code can take significant time, especially for large programs.
- **Debugging:** Debugging can be more difficult because errors are often not found until after compilation.
- **Platform Dependency:** Compiled code is platform-specific, meaning it needs to be recompiled for different operating systems or hardware architectures.

Examples:

- **C/C++:** Programs written in C or C++ are compiled into machine code before execution.

- **Java (Bytecode Compilation):** Java is compiled to bytecode, which is then interpreted or JIT compiled by the JVM.
-

Interpreter

Description: An interpreter translates and executes the source code line-by-line or statement-by-statement at runtime. It does not produce a separate executable file; instead, it executes the code directly.

How It Works:

1. **Source Code:** The programmer writes the code in a high-level language.
2. **Interpretation:** The interpreter translates the source code into machine code line-by-line or statement-by-statement.
3. **Execution:** Each line or statement is executed immediately after translation.

Pros:

- **Immediate Execution:** Interpreted languages allow for immediate execution of the code, which is useful for scripting and rapid prototyping.
- **Ease of Debugging:** Errors are reported as soon as they occur, making debugging easier and more intuitive.
- **Platform Independence:** Interpreted code can often be run on any platform with the appropriate interpreter.

Cons:

- **Performance:** Interpreted programs are usually slower than compiled ones due to the overhead of interpreting each line or statement at runtime.
- **Repetitive Translation:** Each time the program runs, the interpreter must translate the source code, which can be inefficient.
- **Less Optimization:** Interpreters generally perform fewer optimizations compared to compilers.

Examples:

- **Python:** Python code is interpreted line-by-line at runtime.
 - **JavaScript:** JavaScript is typically interpreted by web browsers, allowing for dynamic execution in web pages.
 - **PHP:** PHP is an interpreted language commonly used for server-side scripting.
-

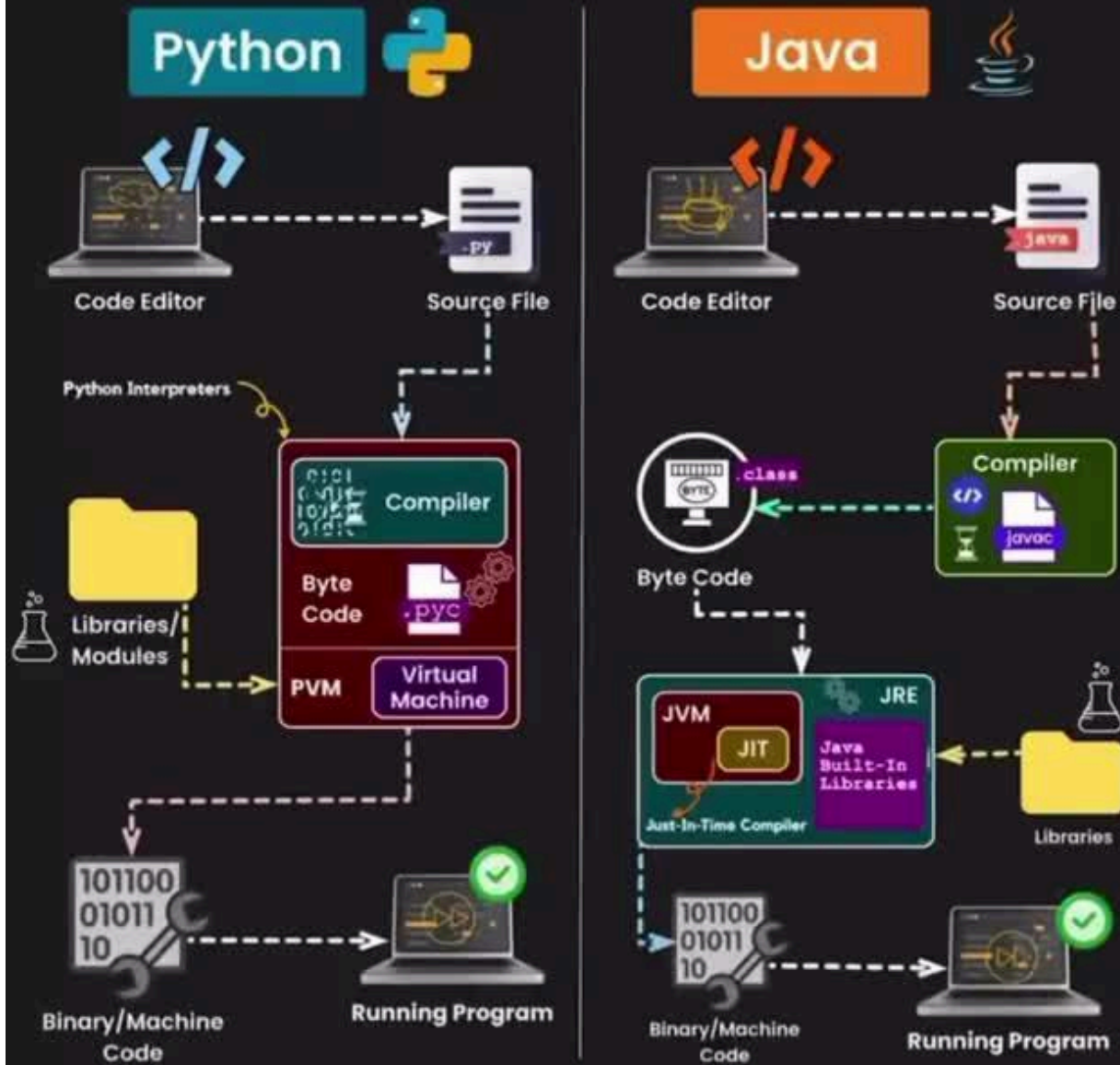
Comparison Table

Feature	Compiler	Interpreter
Translation Method	Translates the entire code before execution	Translates code line-by-line during execution
Execution Speed	Generally faster due to pre-compiled code	Generally slower due to runtime translation
Error Detection	Errors detected at compile-time	Errors detected at runtime
Optimization	High level of optimization	Limited optimization
Platform Dependency	Compiled code is platform-specific	Interpreted code is platform-independent
Use Case	Suitable for performance-critical applications	Suitable for scripting, prototyping, and dynamic applications
Examples	C, C++, Rust, Java (bytecode)	Python, JavaScript, Ruby, PHP

Conclusion

Both compilers and interpreters play crucial roles in software development. Compilers are ideal for applications where performance is critical and where the code does not need to be changed frequently. Interpreters are suitable for scripting languages, rapid development, and scenarios where flexibility and ease of debugging are more important than raw performance. Understanding the differences helps developers choose the right tool for their specific needs.

Working of Python vs Java



Python Tokens and Character Sets

Python is a general-purpose, high-level programming language. It was designed with an emphasis on code readability, and its syntax allows programmers to express their concepts in fewer lines of code, and these codes are known as scripts. These scripts contain character sets, tokens, and identifiers. In this article, we will learn about these character sets, tokens, and identifiers.

Character set

A character set is a set of valid characters acceptable by a programming language in scripting. In this case, we are talking about the Python programming language. So, the Python character set is a valid set of characters recognized by the Python language. These are the characters we can use during writing a script in Python. Python supports all ASCII / Unicode characters that include:

- **Alphabets:** All capital (A-Z) and small (a-z) alphabets.
- **Digits:** All digits 0-9.
- **Special Symbols:** Python supports all kind of special symbols like, " ' | ; : ! ~ @ # \$ % ^ ` & * () _ + - = { } [] \ .
- **White Spaces:** White spaces like tab space, blank space, newline, and carriage return.
- **Other:** All ASCII and UNICODE characters are supported by Python that constitutes the Python character set.

Tokens

A token is the smallest individual unit in a python program. All statements and instructions in a program are built with tokens. The various tokens in python are :

1. Keywords: Keywords are words that have some special meaning or significance in a programming language. They can't be used as variable names, function names, or any other random purpose. They are used for their special features. In Python we have 33 keywords some of them are: *try, False, True, class, break, continue, and, as, assert, while, for, in, raise, except, or, not, if, elif, print, import*, etc.

35 keywords ----- 3.10.5

2. Identifiers: Identifiers are the names given to any variable, function, class, list, methods, etc. for their identification. Python is a case-sensitive language and it has some rules and regulations to name an identifier. Here are some rules to name an identifier:-

- As stated above, Python is case-sensitive. So case matters in naming identifiers. And hence **auratech** and **Auratech** are two different identifiers.
- Identifier starts with a capital letter (A-Z) , a small letter (a-z) or an underscore(_). It can't start with any other character.
- Except for letters and underscore, digits can also be a part of identifier but can't be the first character of it.
- Any other special characters or whitespaces are strictly prohibited in an identifier.
- An identifier can't be a keyword.

3. Literals or Values: Literals are the fixed values or data items used in a source code. Python supports different types of literals such as:

(i) String Literals: The text written in single, double, or triple quotes represents the string literals in Python. For example: "Computer Science", 'sam', etc. We can also use triple quotes to write multi-line strings.

```
# String Literals
a = 'Hello'
b = "Geeks"
c = """Geeks for Geeks is a
      learning platform"""
```

```
# Driver code
print(a)
print(b)
print(c)
```

(ii) Character Literals: Character literal is also a string literal type in which the character is enclosed in single or double-quotes.

```
# Character Literals
a = 'G'
b = "W"
```

```
# Driver code
print(a)
print(b)
```

(iii) Numeric Literals: These are the literals written in the form of numbers. Python supports the following numerical literals:

- **Integer Literal:** It includes both positive and negative numbers along with 0. It doesn't include fractional parts. It can also include binary, decimal, octal, hexadecimal literal.

```
x = 0b10100 #Binary Literals
y = 100 #Decimal Literal
```

```
z = 0o215 #Octal Literal
u = 0x12d #Hexadecimal Literal
```

- **Float Literal:** It includes both positive and negative real numbers. It also includes fractional parts.

```
#Float Literal
float_1 = 100.5
float_2 = 1.5e2
```

- **Complex Literal:** It includes a+bi numeral, here a represents the real part and b represents the complex part.

```
#Complex Literal
a = 5+3.14j
```

(iv) Boolean Literals: Boolean literals have only two values in Python. These are True and False.

```
# Boolean Literals
```

```
a = 3
b = (a == 3)
c = True + 10
```

```
# Driver code
print(a, b, c)
```

(v) Special Literals: Python has a special literal 'None'. It is used to denote nothing, no values, or the absence of value.

```
# Special Literals
var = None
print(var)
```

(vi) Literals Collections: Literals collections in python includes list, tuple, dictionary, and sets.

1. **List:** It is a list of elements represented in square brackets with commas in between. These variables can be of any data type and can be changed as well.
2. **Tuple:** It is also a list of comma-separated elements or values in round brackets. The values can be of any data type but can't be changed.
3. **Dictionary:** It is the unordered set of key-value pairs.
4. **Set:** It is the unordered collection of elements in curly braces '{}'

4. Operators: These are the tokens responsible to perform an operation in an expression. The variables on which operation is applied are called *operands*. Operators can be unary or binary. Unary operators are the ones acting on a single operand like complement operator, etc. While binary operators need two operands to operate.

5. Punctuators: These are the symbols that used in Python to organize the structures, statements, and expressions. Some of the Punctuators are: `[] {} ( ) @ -= += *= /= **== = ,`, etc.

In a Python project, various file extensions are used to organize different types of files that serve different purposes. Here's a list of common file extensions you might encounter in a Python project, along with their descriptions:

1. Python Source Code Files

- **.py**
 - **Description:** Standard Python script files containing source code.
 - **Example:** `main.py`, `utils.py`
- **.pyc**
 - **Description:** Compiled Python files containing bytecode, generated by the Python interpreter to improve loading times.
 - **Example:** `module.pyc`
- **.pyo**
 - **Description:** Optimized bytecode files, generated when Python is run with optimization flags.
 - **Example:** `module.pyo`

2. Python Package Files

- **__init__.py**
 - **Description:** Special file indicating that the directory it resides in is a Python package. Can be empty or contain package initialization code.
 - **Example:** `__init__.py`

3. Configuration Files

- **.ini**
 - **Description:** Configuration files often used for application settings and preferences.
 - **Example:** `config.ini`
- **.cfg**
 - **Description:** General-purpose configuration files, similar to `.ini`.

- **Example:** `setup.cfg`
- **.json**
 - **Description:** JSON (JavaScript Object Notation) files used for configuration, data exchange, or storing structured data.
 - **Example:** `config.json`, `data.json`
- **.yaml / .yml**
 - **Description:** YAML (YAML Ain't Markup Language) files used for configuration, preferred for their readability.
 - **Example:** `config.yaml`, `settings.yml`

4. Data Files

- **.csv**
 - **Description:** Comma-separated values files used to store tabular data.
 - **Example:** `data.csv`
- **.xlsx / .xls**
 - **Description:** Excel spreadsheet files used to store tabular data.
 - **Example:** `data.xlsx`
- **.json**
 - **Description:** JSON files used to store structured data.
 - **Example:** `data.json`
- **.xml**
 - **Description:** XML (eXtensible Markup Language) files used for structured data.
 - **Example:** `data.xml`

5. Documentation Files

- **.md**
 - **Description:** Markdown files used for documentation, such as README files.
 - **Example:** `README.md`, `CONTRIBUTING.md`
- **.rst**
 - **Description:** reStructuredText files used for documentation, often in Python projects.
 - **Example:** `README.rst`

6. Test Files

- **.py**
 - **Description:** Python test files, typically located in a `tests` or `test` directory.
 - **Example:** `test_main.py`, `tests/test_utils.py`

7. Virtual Environment Files

- **requirements.txt**
 - **Description:** Text file listing the dependencies of the project, used with pip.
 - **Example:** requirements.txt
- **Pipfile / Pipfile.lock**
 - **Description:** Files used with Pipenv for dependency management.
 - **Example:** Pipfile, Pipfile.lock
- **pyproject.toml**
 - **Description:** TOML file used to specify build system requirements and package information.
 - **Example:** pyproject.toml

8. Build and Distribution Files

- **setup.py**
 - **Description:** Script for setting up the package, used with setuptools.
 - **Example:** setup.py
- **MANIFEST.in**
 - **Description:** File specifying additional files to include in the source distribution.
 - **Example:** MANIFEST.in
- **.whl**
 - **Description:** Wheel files, a binary distribution format for Python packages.
 - **Example:** package_name-1.0.0-py3-none-any.whl

9. Miscellaneous Files

- **.gitignore**
 - **Description:** File specifying files and directories to ignore in Git version control.
 - **Example:** .gitignore
- **.env**
 - **Description:** File containing environment variables for the project.
 - **Example:** .env
- **.editorconfig**
 - **Description:** Configuration file for defining coding styles between different editors and IDEs.
 - **Example:** .editorconfig

10. Jupyter Notebooks

- **.ipynb**

- **Description:** Jupyter notebook files used for interactive computing, combining code, text, and visualizations.
- **Example:** `analysis.ipynb`

Conclusion

These file extensions help organize and manage different aspects of a Python project, from source code and configuration to documentation and testing. Understanding their roles and purposes is crucial for maintaining a well-structured and efficient Python project.

Python Data Types

Python has a variety of built-in data types that are used to store data in different forms. Understanding these data types is essential for effective programming. In this tutorial, we'll cover the most common data types in Python with examples and exercises.

1. Numeric Types

a. Integer (**int**)

Integers are whole numbers, positive or negative, without a decimal point.

Example:

```
python
```

```
x = 10
y = -5
print(type(x)) # Output: <class 'int'>
print(type(y)) # Output: <class 'int'>
```

Exercise: Create a variable **a** with the value of 20 and another variable **b** with the value of -15. Print their types.

b. Float (**float**)

Floats are numbers that contain a decimal point.

Example:

```
x = 10.5
y = -5.75
print(type(x)) # Output: <class 'float'>
print(type(y)) # Output: <class 'float'>
```

Exercise: Create a variable **a** with the value of 3.14 and another variable **b** with the value of -0.001. Print their types.

c. Complex (**complex**)

Complex numbers are written in the form **a + bj**, where **a** is the real part and **b** is the imaginary part.

Example:

```
x = 1 + 2j
y = 3 - 4j
print(type(x)) # Output: <class 'complex'>
print(type(y)) # Output: <class 'complex'>
```

Exercise: Create a variable **a** with the value of $2 + 3j$ and another variable **b** with the value of $-1 - 1j$. Print their types.

2. Sequence Types

a. String (**str**)

Strings are sequences of characters enclosed in quotes.

Example:

```
x = "Hello, World!"
y = 'Python Programming'
print(type(x)) # Output: <class 'str'>
print(type(y)) # Output: <class 'str'>
```

Exercise: Create a variable **a** with the value "Data Science" and another variable **b** with the value 'Machine Learning'. Print their types.

b. List (**list**)

Lists are ordered collections of items, which can be of different types. Lists are mutable.

Example:

```
x = [1, 2, 3, 4, 5]
y = ["apple", "banana", "cherry"]
print(type(x)) # Output: <class 'list'>
print(type(y)) # Output: <class 'list'>
```

Exercise: Create a list **a** with the values [10, 20, 30] and another list **b** with the values ['x', 'y', 'z']. Print their types.

c. Tuple (**tuple**)

Tuples are ordered collections of items, similar to lists, but they are immutable.

Example:

```
x = (1, 2, 3, 4, 5)
y = ("apple", "banana", "cherry")
print(type(x)) # Output: <class 'tuple'>
print(type(y)) # Output: <class 'tuple'>
```

Exercise: Create a tuple **a** with the values (100, 200, 300) and another tuple **b** with the values ('red', 'green', 'blue'). Print their types.

3. Mapping Types

Dictionary (**dict**)

Dictionaries are collections of key-value pairs. Keys are unique and can be of any immutable type.

Example:

```
x = {"name": "John", "age": 30}
y = {"brand": "Ford", "model": "Mustang", "year": 1964}
print(type(x)) # Output: <class 'dict'>
print(type(y)) # Output: <class 'dict'>
```

Exercise: Create a dictionary **a** with the key-value pairs {"country": "USA", "capital": "Washington, D.C."} and another dictionary **b** with the key-value pairs {"language": "Python", "version": 3.9}. Print their types.

4. Set Types

Set (**set**)

Sets are unordered collections of unique items.

Example:

```
x = {1, 2, 3, 4, 5}
y = {"apple", "banana", "cherry"}
print(type(x)) # Output: <class 'set'>
print(type(y)) # Output: <class 'set'>
```

Exercise: Create a set **a** with the values {10, 20, 30, 40} and another set **b** with the values {'alpha', 'beta', 'gamma'}. Print their types.

Frozen Set (**frozenset**)

Frozen sets are immutable sets.

Example:

```
x = frozenset([1, 2, 3, 4, 5])
y = frozenset(["apple", "banana", "cherry"])
print(type(x)) # Output: <class 'frozenset'>
print(type(y)) # Output: <class 'frozenset'>
```

Exercise: Create a frozen set **a** with the values {100, 200, 300} and another frozen set **b** with the values {'red', 'green', 'blue'}. Print their types.

5. Boolean Type

Boolean (**bool**)

Booleans represent one of two values: **True** or **False**.

Example:

```
x = True
y = False
print(type(x)) # Output: <class 'bool'>
print(type(y)) # Output: <class 'bool'>
```

Exercise: Create a variable **a** with the value **True** and another variable **b** with the value **False**. Print their types.

6. None Type

NoneType (**None**)

None is used to define a null value or a lack of value.

Example:

```
x = None
print(type(x)) # Output: <class 'NoneType'>
```

Exercise: Create a variable `a` with the value `None`. Print its type.

Summary

Understanding these data types and their usage is crucial for writing effective and efficient Python programs. Let's practice with a few exercises to solidify your understanding.

Exercises

- Variables and Types:**
 - Create variables of each data type discussed above and print their types.
- List Operations:**
 - Create a list of five different fruits. Add a new fruit to the list, remove one fruit, and print the final list.
- Dictionary Operations:**
 - Create a dictionary with information about a book (title, author, year). Update the year and add a new key-value pair for the genre. Print the final dictionary.
- Set Operations:**
 - Create a set of three programming languages. Add a new language to the set and remove one language. Print the final set.
- Boolean and None:**
 - Create a boolean variable indicating if a user is logged in. Create another variable set to `None` to represent an uninitialized state. Print both variables.

By completing these exercises, you'll gain a solid understanding of Python's data types and how to work with them in your programs.

Operators in Python

Python supports various types of operators that are used to perform operations on variables and values. In this tutorial, we'll cover the different types of operators available in Python, provide examples, and offer exercises to reinforce understanding.

1. Arithmetic Operators

Arithmetic operators are used to perform basic mathematical operations.

Operator	Description	Example	Result
+	Addition	<code>x + y</code>	Sum of <code>x</code> and <code>y</code>
-	Subtraction	<code>x - y</code>	Difference of <code>x</code> and <code>y</code>
*	Multiplication	<code>x * y</code>	Product of <code>x</code> and <code>y</code>
/	Division	<code>x / y</code>	Quotient of <code>x</code> and <code>y</code>
%	Modulus	<code>x % y</code>	Remainder of <code>x</code> divided by <code>y</code>
**	Exponentiation	<code>x ** y</code>	<code>x</code> raised to the power of <code>y</code>
//	Floor Division	<code>x // y</code>	Quotient of <code>x</code> and <code>y</code> (floored)

Examples:

```
x = 10
y = 3
```

```
print(x + y) # Output: 13
print(x - y) # Output: 7
print(x * y) # Output: 30
print(x / y) # Output: 3.3333333333333335
print(x % y) # Output: 1
print(x ** y) # Output: 1000
print(x // y) # Output: 3
```

Exercise:

- Create two variables **a** and **b** with values 15 and 4. Perform all arithmetic operations on these variables and print the results.
-

2. Comparison Operators

Comparison operators are used to compare two values. They return a Boolean value (**True** or **False**).

Operator	Description	Example	Result
==	Equal	x == y	True if x is equal to y
!=	Not Equal	x != y	True if x is not equal to y
>	Greater than	x > y	True if x is greater than y
<	Less than	x < y	True if x is less than y
>=	Greater than or equal	x >= y	True if x is greater than or equal to y
<=	Less than or equal	x <= y	True if x is less than or equal to y

Examples:

```
x = 10
y = 3
```

```
print(x == y) # Output: False
print(x != y) # Output: True
print(x > y)  # Output: True
print(x < y)  # Output: False
print(x >= y) # Output: True
print(x <= y) # Output: False
```

Exercise:

- Create two variables **a** and **b** with values 25 and 30. Perform all comparison operations on these variables and print the results.

3. Logical Operators

Logical operators are used to combine conditional statements.

Operator	Description	Example	Result
and	Logical AND	<code>x > 5 and y < 10</code>	True if both conditions are true
or	Logical OR	<code>x > 5 or y < 10</code>	True if at least one condition is true
not	Logical NOT	<code>not(x > 5 and y < 10)</code>	True if the condition is false

Examples:

```
x = 10
```

```
y = 3
```

```
print(x > 5 and y < 10) # Output: True
```

```
print(x > 15 or y < 10) # Output: True
```

```
print(not(x > 5 and y < 10)) # Output: False
```

Exercise:

- Create two variables `a` and `b` with values 8 and 12. Use logical operators to check various conditions and print the results.

4. Bitwise Operators

Bitwise operators are used to perform bit-level operations.

Operator	Description	Example	Result
&	AND	<code>x & y</code>	Bitwise AND
	OR	<code>x y</code>	Bitwise OR
^	XOR	<code>x ^ y</code>	Bitwise XOR

~	NOT	~x	Bitwise NOT
<<	Left Shift	x << 2	Bitwise left shift
>>	Right Shift	x >> 2	Bitwise right shift

Examples:

x = 10 # Binary: 1010

y = 4 # Binary: 0100

print(x & y) # Output: 0 (Binary: 0000)

print(x | y) # Output: 14 (Binary: 1110)

print(x ^ y) # Output: 14 (Binary: 1110)

print(~x) # Output: -11 (Binary: 1011 in two's complement)

print(x << 2) # Output: 40 (Binary: 101000)

print(x >> 2) # Output: 2 (Binary: 0010)

Exercise:

- Create two variables **a** and **b** with values 7 and 3. Perform all bitwise operations on these variables and print the results.

5. Assignment Operators

Assignment operators are used to assign values to variables.

Operator	Description	Example	Result
=	Assign	x = 5	x is assigned 5
+=	Add and assign	x += 3	x is incremented by 3
-=	Subtract and assign	x -= 2	x is decremented by 2
*=	Multiply and assign	x *= 3	x is multiplied by 3
/=	Divide and assign	x /= 2	x is divided by 2
%=	Modulus and assign	x %= 2	x is assigned the remainder of x divided by 2

**=	Exponent and assign	x **= 3	x is raised to the power of 3
//=	Floor divide and assign	x //= 2	x is floored after division by 2
&=	Bitwise AND and assign	x &= 3	x is assigned the result of x & 3
 =	Bitwise OR and assign	x = 3	x is assigned the result of x 3
^=	Bitwise XOR and assign	x ^= 3	x is assigned the result of x ^ 3
>>=	Bitwise right shift and assign	x >>= 2	x is right shifted by 2 and assigned
<<=	Bitwise left shift and assign	x <<= 2	x is left shifted by 2 and assigned

Examples:

```
x = 5
x += 3
print(x) # Output: 8
```

```
x = 5
x *= 3
print(x) # Output: 15
```

Exercise:

- Create a variable **a** with the value 10. Use all assignment operators on this variable and print the results.

6. Membership Operators

Membership operators are used to test if a sequence contains a specified item.

Operator	Description	Example	Result
in	Membership	5 in [1, 2, 3, 4, 5]	True if 5 is in the list

`not in` Not Membership `6 not in [1, 2, 3, 4, 5]` `True` if 6 is not in the list

Examples:

```
x = [1, 2, 3, 4, 5]
print(3 in x)    # Output: True
print(6 not in x) # Output: True
```

Exercise:

- Create a list `a` with the values `[10, 20, 30, 40, 50]`. Use membership operators to check if 20 is in the list and if 25 is not in the list. Print the results.
-

7. Identity Operators

Identity operators are used to compare the memory locations of two objects.

Operator	Description	Example	Result
<code>is</code>	Identity	<code>x is y</code>	<code>True</code> if <code>x</code> and <code>y</code> are the same object
<code>is not</code>	Not Identity	<code>x is not y</code>	<code>True</code> if <code>x</code> and <code>y</code> are not the same object

Examples:

```
x = [1, 2, 3]
y = [1, 2, 3]
z = x

print(x is y)    # Output: False
print(x is z)    # Output: True
print(x is not y) # Output: True
```

Exercise:

- Create two lists `a` and `b` with the same values `[1, 2, 3]`. Create another variable `c` that references `a`. Use identity operators to compare `a`, `b`, and `c`. Print the results.

Python - Arithmetic Operators

In Python, numbers are the most frequently used [data type](#). Python uses the same symbols for basic arithmetic operations Everybody is familiar with, i.e., "+" for addition, "-" for subtraction, "*" for multiplication (most programming languages use "*" instead of the "x" as used in maths/algebra), and "/" for division (again for the "÷" used in Mathematics).

In addition, Python defines few more arithmetic operators. They are "%" (Modulus), "**" (Exponent) and "//" (Floor division).

Python Arithmetic Operators

Arithmetic operators are binary operators in the sense they operate on two operands. Python fully supports mixed arithmetic. That is, the two operands can be of two different number types. In such a situation, Python widens the narrower of the operands. An integer object is narrower than float object, and float is narrower than complex object. Hence, the result of arithmetic operation of int and a float is a float. Result of float and a complex is a complex number, similarly, operation on an integer and a complex object results in a complex object.

Different Arithmetic Operators in Python

Following is the table which lists down all the arithmetic operators available in Python:

Operat or	Name	Example
+	Addition	$a + b = 30$

-	Subtraction	$a - b = -10$
*	Multiplication	$a * b = 200$
/	Division	$b / a = 2$
%	Modulus	$b \% a = 0$
**	Exponent	$a ** b = 10 ** 20$
//	Floor Division	$9 // 2 = 4$

Let us study these operators with examples.

Addition Operator (+)

This operator pronounced as plus, is a basic arithmetic operator. It adds the two numeric operands on the either side and returns the addition result.

Example: Addition of Two Integers

In the following example, the two integer [variables](#) are the operands for the "+" operator.

```
a=10
```

```
b=20
```

```
print ("Addition of two integers")
```

```
print ("a =",a,"b =",b,"addition =",a+b)
```

Summary

Understanding these operators and their usage is essential for writing effective and efficient Python programs. Let's practice with a few exercises to solidify your understanding.

Exercises

- 1. Arithmetic Operations:**
 - Create two variables `a` and `b` with values 50 and 5. Perform all arithmetic operations on these variables and print the results.
- 2. Comparison Operations:**
 - Create two variables `x` and `y` with values 100 and 50. Perform all comparison operations on these variables and print the results.
- 3. Logical Operations:**
 - Create two boolean variables `a` and `b` with values `True` and `False`. Use logical operators to combine these variables and print the results.
- 4. Bitwise Operations:**
 - Create two variables `m` and `n` with values 9 and 4. Perform all bitwise operations on these variables and print the results.
- 5. Assignment Operations:**
 - Create a variable `p` with the value 20. Use all assignment operators on this variable and print the results.
- 6. Membership Operations:**
 - Create a string `s` with the value "Python". Use membership operators to check if the letter "P" is in the string and if the letter "z" is not in the string. Print the results.
- 7. Identity Operations:**
 - Create two tuples `t1` and `t2` with the same values (1, 2, 3). Create another variable `t3` that references `t1`. Use identity operators to compare `t1`, `t2`, and `t3`. Print the results.

By completing these exercises, you'll gain a solid understanding of Python's operators and how to use them effectively in your programs.

Python Conditions and If statements

